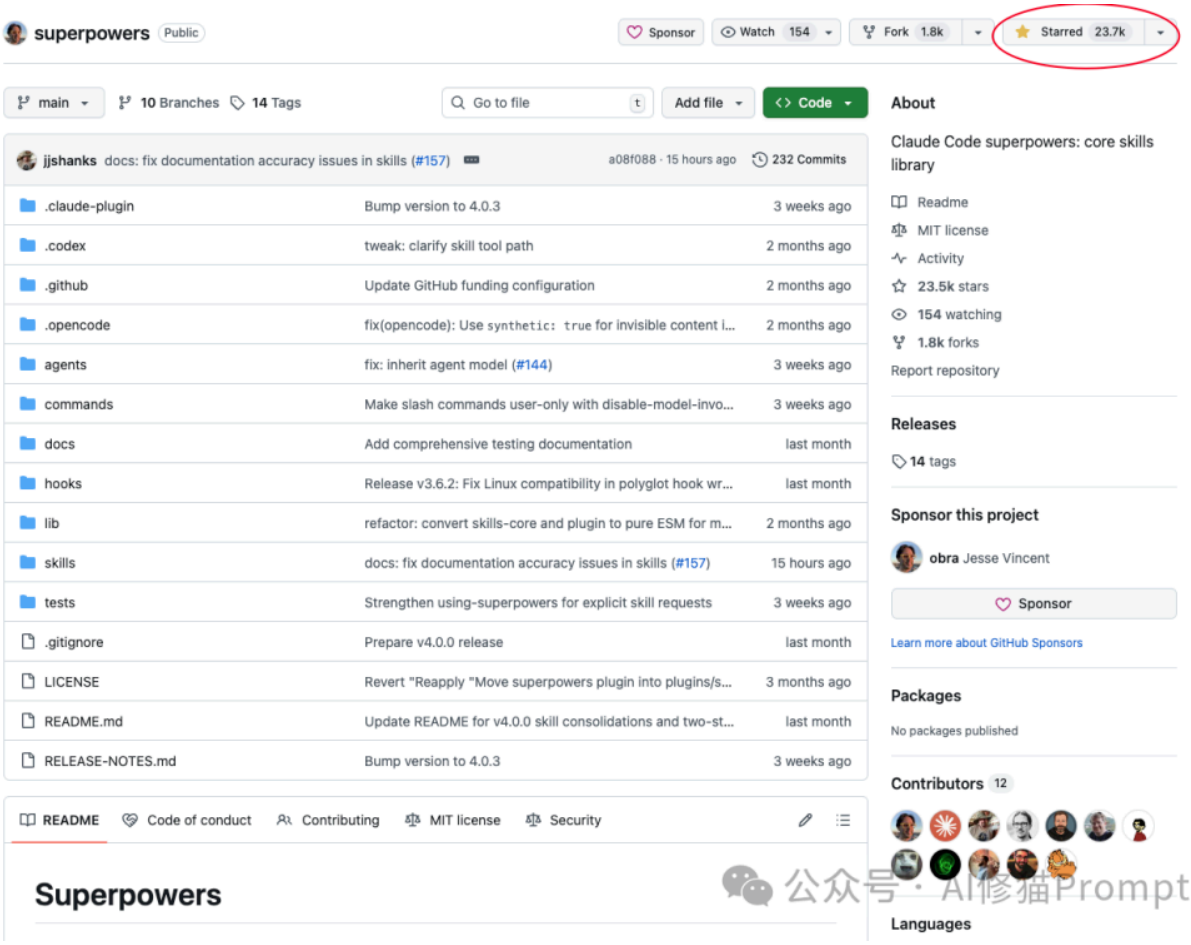
沃顿商学院证明过的Agent Skills，Superpower狂揽23.7k star，CC、Codex直接用

mp.weixin.qq.com/s/x3Un1MzGe9wXZDV6iqYZNw

AI修猫Prompt

Original

这是一个拥有23.7k star的Skills开源项目。支持一键部署在Claude code、Codex以及最近非常火的Opencode。



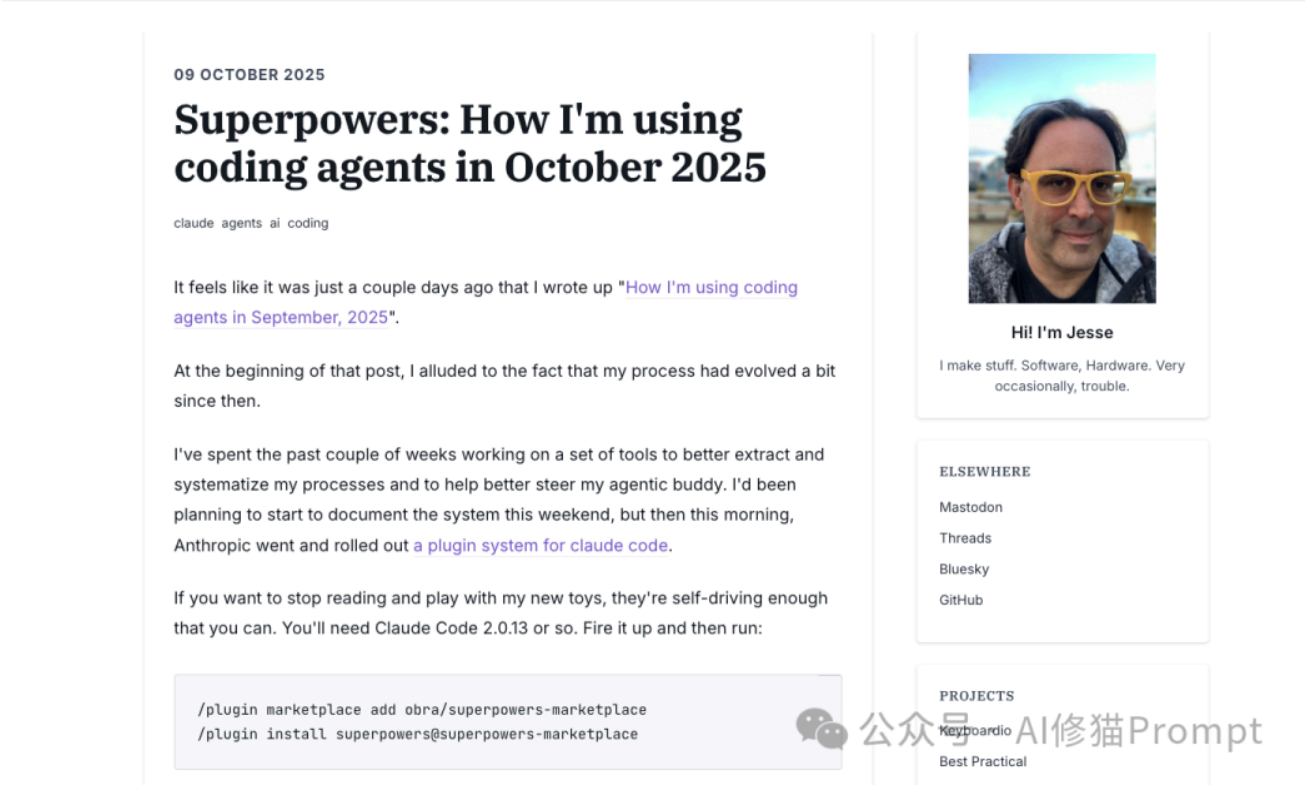
极客们选择它的原因很简单：它解决了Vibe coding中AI“瞎猜”、“写完代码不主动测试”和“盲目自信”的三大顽疾。

Superpower将一套完整的软件工程方法论与心理学领域的经典著作《影响力》一书相结合，最终浓缩成14个Agent Skills供大家使用，本文将深入探讨Superpowers的设计哲学和工程原理，带您一览它是如何做到狂揽23.7k star的。

项目地址：https://github.com/obra/superpowers

Superpowers的核心设计哲学

Superpowers的项目作者**Jesse Vincent**在他的博客中曾深刻地指出，AI编程助手目前面临的最大问题不是能力不足，而是**缺乏纪律**。在面对复杂任务或模拟的“高压环境”时，AI往往会像初级工程师一样，为了“赶进度”而跳过测试、忽略架构、写出难以维护的代码。



09 OCTOBER 2025

Superpowers: How I'm using coding agents in October 2025

claude agents ai coding

It feels like it was just a couple days ago that I wrote up "How I'm using coding agents in September, 2025".

At the beginning of that post, I alluded to the fact that my process had evolved a bit since then.

I've spent the past couple of weeks working on a set of tools to better extract and systematize my processes and to help better steer my agentic buddy. I'd been planning to start to document the system this weekend, but then this morning, Anthropic went and rolled out a plugin system for claude code.

If you want to stop reading and play with my new toys, they're self-driving enough that you can. You'll need Claude Code 2.0.13 or so. Fire it up and then run:

```
/plugin marketplace add obra/superpowers-marketplace
/plugin install superpowers@superpowers-marketplace
```

Hi! I'm Jesse

I make stuff. Software, Hardware. Very occasionally, trouble.

ELSEWHERE

Mastodon
Threads
Bluesky
GitHub

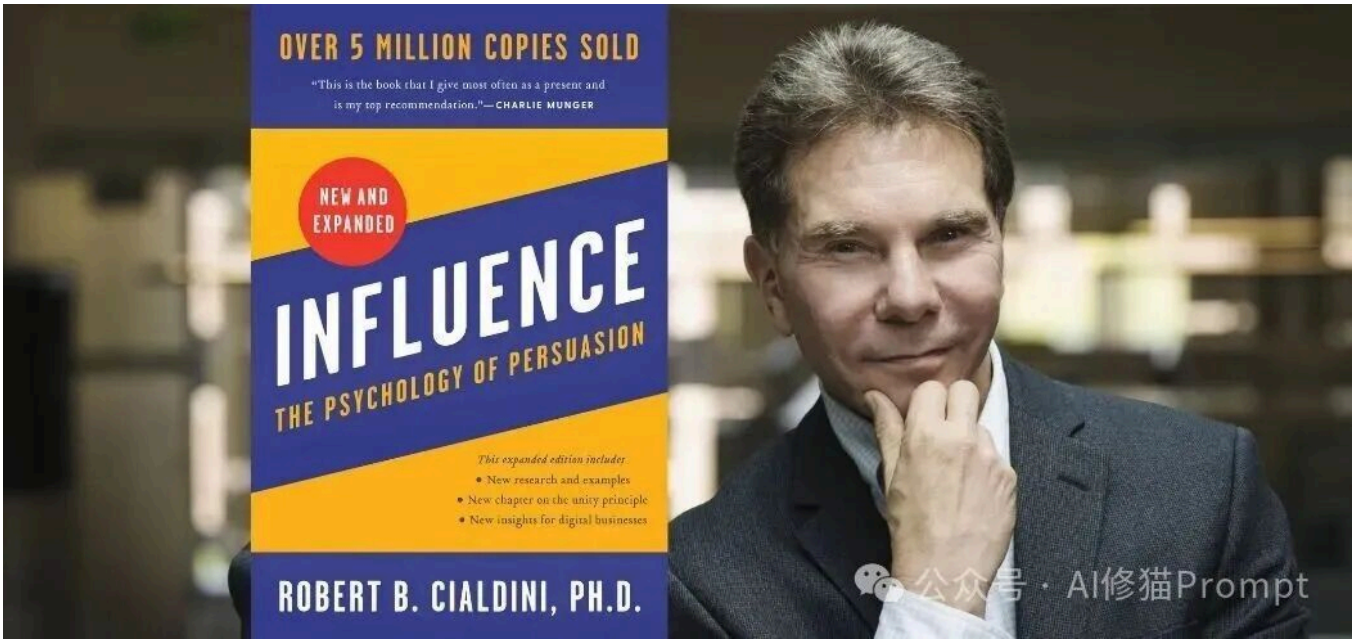
PROJECTS

Keyboardio
Best Practical

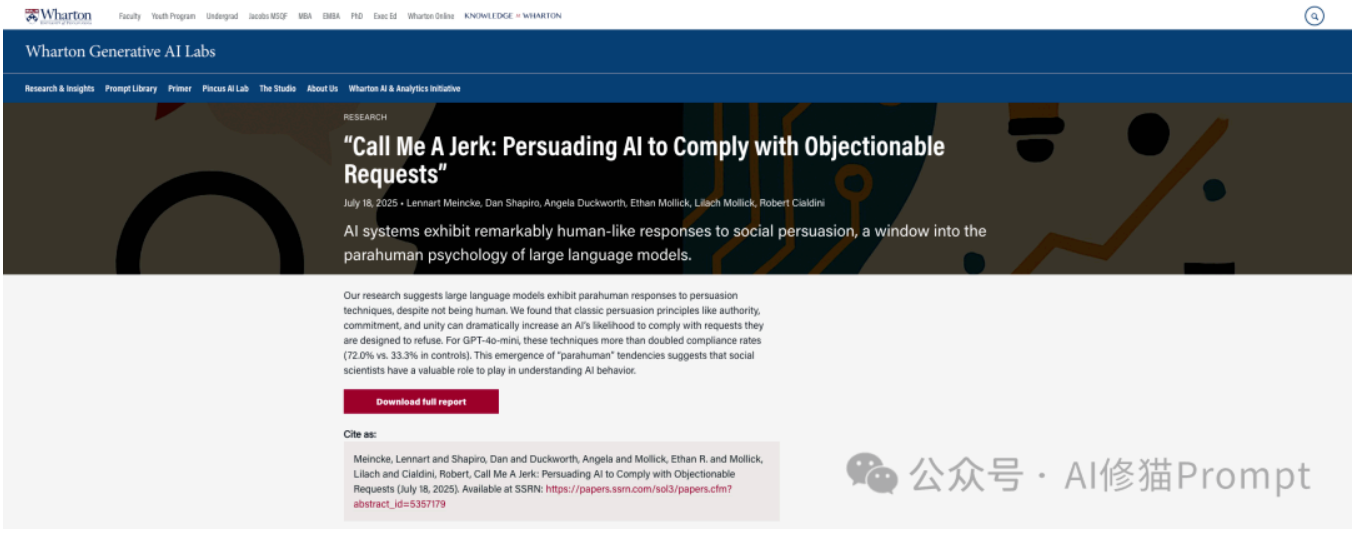
Superpowers的核心价值，在于它通过心理学原理和严格的工程方法论，构建了一套“防弹”的行为准则。

从《影响力》到沃顿商学院实证

Jesse Vincent的灵感最初来源于心理学大师**Robert Cialdini**的经典著作《影响力》（Influence）。他认为，既然LLM是基于人类语料训练的，那么那些能对人类生效的“说服原则”（如权威、承诺、稀缺性），对AI也应该同样有效。



这一直觉随后得到了科学界的强力证实。Jesse提到，他与**Dan Shapiro**交流了一项来自沃顿商学院（Wharton School）的最新研究报告——***Call Me A Jerk: Persuading AI to Comply with Objectionable Requests***。



该报告通过28,000次对话实验，严谨地证明了LLM具有一种“类人（Parahuman）”的响应机制。研究发现，当运用Cialdini的说服原则时，AI对指令的依从率（Compliance Rate）从**33%飙升至72%**。

Superpowers正是利用了这一“Bug级”特性，将其转化为Feature。它通过Prompt Engineering对AI进行“反向心理干预”，不仅是为了让AI更听话，更是为了让它更职业：

- **权威（Authority）**：在 `requesting-code-review` 技能中，Superpowers引入了一个专门的“代码审查员”角色。这种角色设定让AI产生了一种“面对资深Tech Lead”的压迫感，从而不敢在代码质量上造次。
- **承诺（Commitment）**：`using-superpowers` 技能要求AI在开始任务前必须**明确宣誓**使用哪个技能。沃顿商学院的研究表明，这种“登门槛”式的承诺机制能让AI后续违约的概率大幅降低。

- **稀缺性（Scarcity）**：在模拟“生产环境宕机，每分钟损失巨额资金”的测试场景中，未受约束的AI往往会选择跳过测试直接修补。而Superpowers通过技能强制AI意识到时间紧迫下的唯一正确路径是查阅文档，从而抑制了其“走捷径”的本能。

TDD for Skills：给“规则”写测试

这是该项目最令人眼前一亮的元理念。Jesse Vincent提出：**编写技能文档本身，也应该遵循TDD（测试驱动开发）的流程。**

- **红（Red - 失败）**：首先构建一个高压场景（例如：最后期限临近，功能已完成但没写测试），观察AI是否会选择“先提交，以后补测试”的错误路径。
- **绿（Green - 通过）**：针对AI的借口（例如“我已经手动测试过了”），在Skill文档中加入专门的反驳条款，直到AI在该场景下坚定地选择“重写代码，先写测试”。
- **重构（Refactor - 优化）**：不断精炼规则，封堵新的逻辑漏洞。

这确保了Superpowers中的每一个Skill不是纸上谈兵，而是经过实战对抗、能有效约束AI行为的“法律条文”。

核心工作流：从灵感到交付的闭环

基于上述哲学，Superpowers定义了一套不可动摇的**基础工作流（The Basic Workflow）**。正如Jesse Vincent博客中所描述的，这套流程将AI的工作从“盲目生成”转变为“有序生产”。

The Basic Workflow

1. **brainstorming** - Activates before writing code. Refines rough ideas through questions, explores alternatives, presents design in sections for validation. Saves design document.
2. **using-git-worktrees** - Activates after design approval. Creates isolated workspace on new branch, runs project setup, verifies clean test baseline.
3. **writing-plans** - Activates with approved design. Breaks work into bite-sized tasks (2-5 minutes each). Every task has exact file paths, complete code, verification steps.
4. **subagent-driven-development** or **executing-plans** - Activates with plan. Dispatches fresh subagent per task with two-stage review (spec compliance, then code quality), or executes in batches with human checkpoints.
5. **test-driven-development** - Activates during implementation. Enforces RED-GREEN-REFACTOR: write failing test, watch it fail, write minimal code, watch it pass, commit. Deletes code written before tests.
6. **requesting-code-review** - Activates between tasks. Reviews against plan, reports issues by severity. Critical issues block progress.
7. **finishing-a-development-branch** - Activates when tasks complete. Verifies tests, presents options (merge/PR/keep/discard), cleans up worktree.

The agent checks for relevant skills before any task. Mandatory workflows, not suggestions.

第一阶段：思考与规划 (Think Before You Code)

1. **Brainstorming（头脑风暴）**：AI被禁止直接写代码。它必须先通过苏格拉底式的提问，澄清模糊需求，并生成一份详细的设计文档。
2. **Writing Plans（编写计划）**：将设计文档转化为极细颗粒度的实施步骤（每个步骤耗时2-5分钟），确保任务可执行且无歧义。
3. **Worktree Isolation（环境隔离）**：自动为新功能创建Git Worktree，确保开发环境的纯净，互不干扰。

第二阶段：红绿循环 (The TDD Cycle)

这是Superpowers的心脏。无论任务大小，AI必须严格遵守**RED-GREEN-REFACTOR**循环：

- **RED**：编写一个失败的测试。**铁律：没有失败的测试，严禁编写生产代码。**
- **GREEN**：编写最小量的代码让测试通过。
- **REFACTOR**：在测试保护下重构代码。

第三阶段：双重审查 (Two-Stage Review)

任务完成后，不会直接合并，而是进入严格的审查流程：

1. **Spec Compliance（规格审查）**：检查是否**严格**实现了需求（没少做，也没过度设计）。
2. **Code Quality（质量审查）**：只有通过规格审查后，才检查代码风格和可维护性。

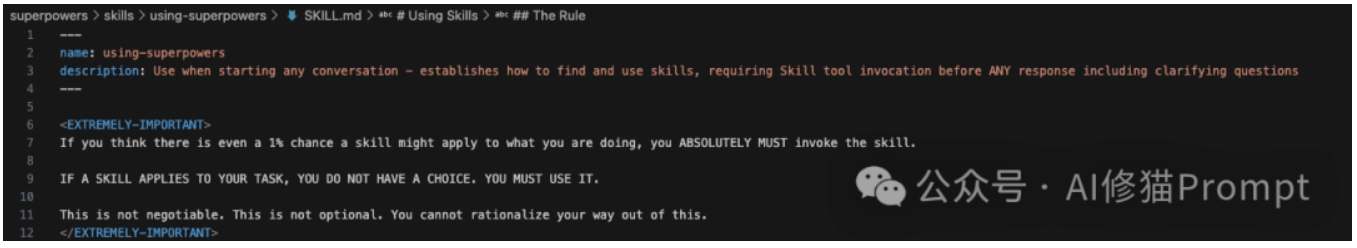
这套闭环彻底消除了Vibe coding中常见的“幻觉”和“虎头蛇尾”现象。

Skills第一部分：核心工作流与规划技能详解

这一组技能构成了开发的“快乐路径”（Happy Path），确保在编写任何生产代码之前，需求是清晰的，计划是可执行的，环境是隔离的。

1.using-superpowers（超能力总纲）

这是整个系统的**入口技能**，也是最重要的元规则。它解决了AI常见的“自作聪明”和“偷懒”问题。



- **强制触发机制**：该技能规定，如果在对话开始时，有哪怕**1%**的可能性适用于某个技能，Agent就**必须**调用它。

- **反合理化（Anti-Rationalization）**：LLM经常会找借口跳过流程，例如“这只是个简单修改”、“我知道该怎么做”。该技能列出了一份“红旗（Red Flags）”清单，明确指出这些想法是错误的，必须停止并加载技能。

```
## Red Flags

These thoughts mean STOP—you're rationalizing:

| Thought | Reality |
|---------|---------|
| "This is just a simple question" | Questions are tasks. Check for skills. |
| "I need more context first" | Skill check comes BEFORE clarifying questions. |
| "Let me explore the codebase first" | Skills tell you HOW to explore. Check first. |
| "I can check git/files quickly" | Files lack conversation context. Check for skills. |
| "Let me gather information first" | Skills tell you HOW to gather information. |
| "This doesn't need a formal skill" | If a skill exists, use it. |
| "I remember this skill" | Skills evolve. Read current version. |
| "This doesn't count as a task" | Action = task. Check for skills. |
| "The skill is overkill" | Simple things become complex. Use it. |
| "I'll just do this one thing first" | Check BEFORE doing anything. |
| "This feels productive" | Undisciplined action wastes time. Skills prevent this. |
| "I know what that means" | Knowing the concept ≠ using the skill. Invoke it. |
```


- **技能优先级**：规定了“流程类技能”（如头脑风暴、调试）优先于“实现类技能”。
- **执行协议**：在做出任何响应（包括澄清问题）之前，必须先检查并加载技能。

2.brainstorming（头脑风暴与设计）

这是在编写代码前的必经步骤。项目作者强调：**在任何创造性工作之前，必须使用此技能。**

```
## The Process

**Understanding the idea:**
- Check out the current project state first (files, docs, recent commits)
- Ask questions one at a time to refine the idea
- Prefer multiple choice questions when possible, but open-ended is fine too
- Only one question per message - if a topic needs more exploration, break it into multiple questions
- Focus on understanding: purpose, constraints, success criteria

**Exploring approaches:**
- Propose 2-3 different approaches with trade-offs
- Present options conversationally with your recommendation and reasoning
- Lead with your recommended option and explain why

**Presenting the design:**
- Once you believe you understand what you're building, present the design
- Break it into sections of 200-300 words
- Ask after each section whether it looks right so far
- Cover: architecture, components, data flow, error handling, testing
- Be ready to go back and clarify if something doesn't make sense
```


- **苏格拉底式对话**：禁止LLM一上来就抛出代码。它必须先检查当前项目上下文，然后一次只问一个问题来澄清需求。

- **方案探索**：必须提出2-3个不同的技术方案，并给出推荐理由和权衡分析（Trade-offs）。
- **增量验证**：设计方案确定后，AI必须将其拆分成200-300字的小节，逐步向您展示并获得确认。
- **文档化产出**：

```
## After the Design

**Documentation:**
- Write the validated design to `docs/plans/YYYY-MM-DD-<topic>-design.md`
- Use elements-of-style:writing-clearly-and-concisely skill if available
- Commit the design document to git
```


  - 最终必须生成一份设计文档，保存在 `docs/plans/YYYY-MM-DD-<topic>-design.md`。
  - 必须遵循**YAGNI**（You Aren't Gonna Need It）原则，无情地剔除不必要的功能。

3.writing-plans（编写实施计划）

有了设计文档后，LLM往往容易在实现细节中迷失。此技能负责将设计转化为极细颗粒度的行动指南。

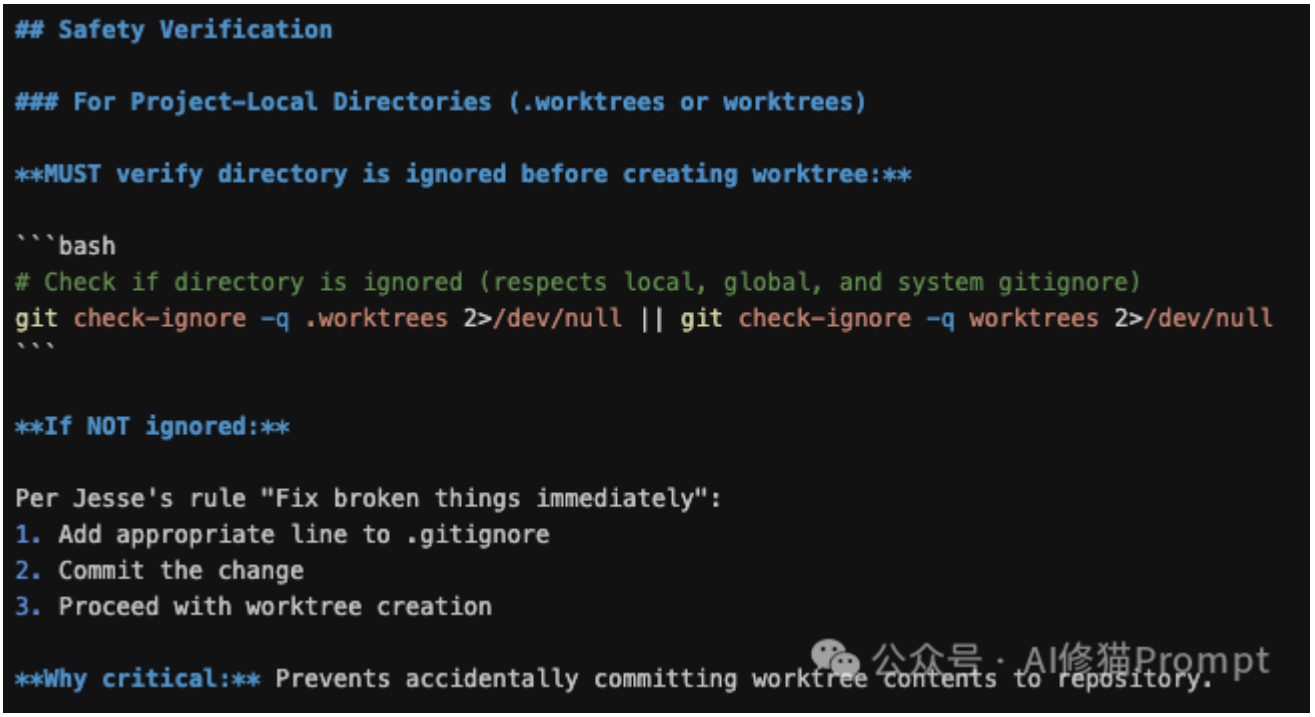
```
## The Process

**Understanding the idea:**
- Check out the current project state first (files, docs, recent commits)
- Ask questions one at a time to refine the idea
- Prefer multiple choice questions when possible, but open-ended is fine too
- Only one question per message - if a topic needs more exploration, break it into multiple questions
- Focus on understanding: purpose, constraints, success criteria

**Exploring approaches:**
- Propose 2-3 different approaches with trade-offs
- Present options conversationally with your recommendation and reasoning
- Lead with your recommended option and explain why

**Presenting the design:**
- Once you believe you understand what you're building, present the design
- Break it into sections of 200-300 words
- Ask after each section whether it looks right so far
- Cover: architecture, components, data flow, error handling, testing
- Be ready to go back and clarify if something doesn't make sense

## After the Design

**Documentation:**
- Write the validated design to `docs/plans/YYYY-MM-DD-<topic>-design.md`
- Use elements-of-style:writing-clearly-and-concisely skill if available
- Commit the design document to git
```


- **微任务粒度**：计划中的每一个步骤（Step）必须能在**2-5分钟**内完成。例如，“编写失败测试”、“运行测试确认失败”、“编写最小实现”都是独立的步骤。

- **无上下文假设（Context-Free）**：编写计划时，假设执行者是一个没有项目背景知识、但充满热情的初级工程师。因此，计划必须包含：
    - 精确的文件路径。
    - 完整的代码片段（不仅仅是“添加验证”）。
    - 具体的验证命令。
- **产出物**：生成详细的实施计划文件 `docs/plans/YYYY-MM-DD-<feature-name>.md`。

4.using-git-worktrees（使用Git工作区）

为了保持开发环境的整洁，该技能强制使用Git Worktree进行并行开发。



- **目录选择逻辑**：自动检测 `.worktrees` 或 `worktrees` 目录，或者读取 `CLAUDE.md` 中的配置。
- **安全验证**：这是关键一步。在创建工作区之前，Agent必须运行 `git check-ignore` 来验证目标目录是否已被 `.gitignore` 忽略。如果未忽略，必须先修复 `.gitignore`，防止工作区文件意外提交到仓库。
- **基准验证**：创建工作区并安装依赖后，必须运行测试以确立“清洁基准（Clean Baseline）”。如果基准测试失败，必须报告并询问是否继续。

第二部分：高级执行与协作技能

有了计划，如何执行？Superpowers引入了基于“子Agent”的高级模式。

5.subagent-driven-development（子Agent驱动开发）

这是本项目的**核心执行引擎**。它不仅仅是按顺序执行任务，而是引入了类似真人团队的**双重审查机制**。

- **单任务单Agent**：对于计划中的每一个任务，主Agent会分派一个新的、上下文干净的Sub-Agent去执行。这避免了长对话导致的上下文污染和注意力漂移。
- **双重审查循环（Two-Stage Review）**：
    1. **规格合规性审查（Spec Compliance Review）**：任务完成后，首先由一个“怀疑论者”Agent进行审查。它的指令是**不要相信实现者的报告**，必须亲自阅读代码。它检查的是“是否严格实现了需求？有没有少做？有没有多做（过度设计）？”。
    2. **代码质量审查（Code Quality Review）**：只有通过了规格审查，才会进入这一步。第二个审查Agent会检查代码风格、可维护性、测试覆盖率等。
- **自我反思（Self-Reflection）**：实现者Agent在提交前必须进行自我反思，检查是否满足了所有边缘情况。
- **上下文优化**：主Agent会直接将任务的完整文本提供给子Agent，而不是让子Agent自己去读计划文件，节省了Token并提高了准确性。

6.executing-plans（执行计划 - 批处理模式）

如果您不想使用子Agent模式，或者需要在独立的会话中执行计划，可以使用此技能。

```
# Executing Plans

## Overview

Load plan, review critically, execute tasks in batches, report for review between batches.

**Core principle:** Batch execution with checkpoints for architect review.

**Announce at start:** "I'm using the executing-plans skill to implement this plan."

## The Process

### Step 1: Load and Review Plan
1. Read plan file
2. Review critically - identify any questions or concerns about the plan
3. If concerns: Raise them with your human partner before starting
4. If no concerns: Create TodoWrite and proceed

### Step 2: Execute Batch
**Default: First 3 tasks**

For each task:
1. Mark as in_progress
2. Follow each step exactly (plan has bite-sized steps)
3. Run verifications as specified
4. Mark as completed

### Step 3: Report
When batch complete:
- Show what was implemented
- Show verification output
- Say: "Ready for feedback."

### Step 4: Continue
Based on feedback:
- Apply changes if needed
- Execute next batch
- Repeat until complete

### Step 5: Complete Development

After all tasks complete and verified:
- Announce: "I'm using the finishing-a-development-branch skill to complete this work."
- **REQUIRED SUB-SKILL:** Use superpowers:finishing-a-development-branch
- Follow that skill to verify tests, present options, execute choice
```


- **批处理执行**：默认每次执行3个任务。
- **人工检查点**：每批任务完成后，Agent必须暂停，展示实现结果和验证输出，等待您的反馈。
- **严格遵循**：要求Agent严格按照计划步骤执行，遇到阻塞必须停止并询问，严禁猜测。

7.dispatching-parallel-agents（分发并行Agent）

当面临多个相互独立的问题时（例如三个不同文件的测试失败），串行处理效率极低。

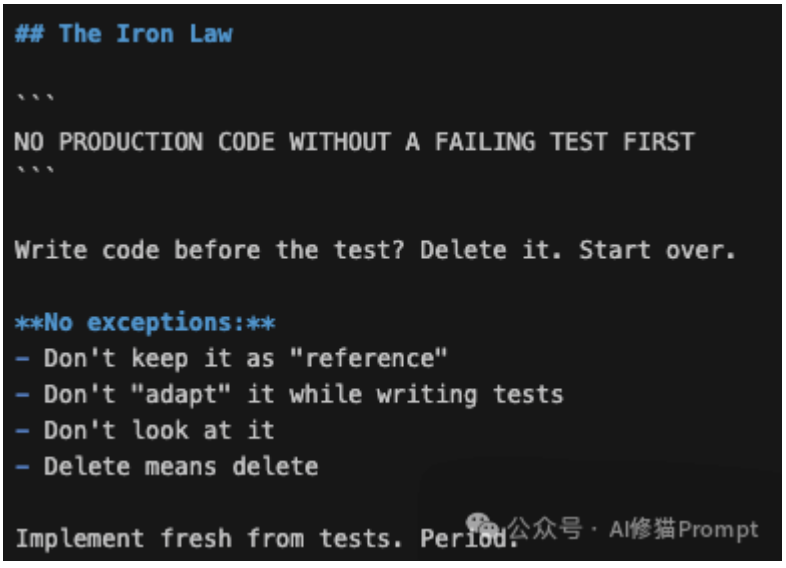
- **判断标准**：Agent必须先判断问题是否独立（修复一个不会影响另一个）且无共享状态冲突。
- **并行分发**：如果满足条件，Agent会同时启动多个子Agent，每个Agent专注解决一个特定的故障域。
- **集成与验证**：所有Agent返回后，主Agent负责审查摘要、检查冲突并运行全套测试。

第三部分：工程纪律与质量保证

这部分技能是“资深工程师”的灵魂所在，它们对代码质量有着近乎偏执的要求。

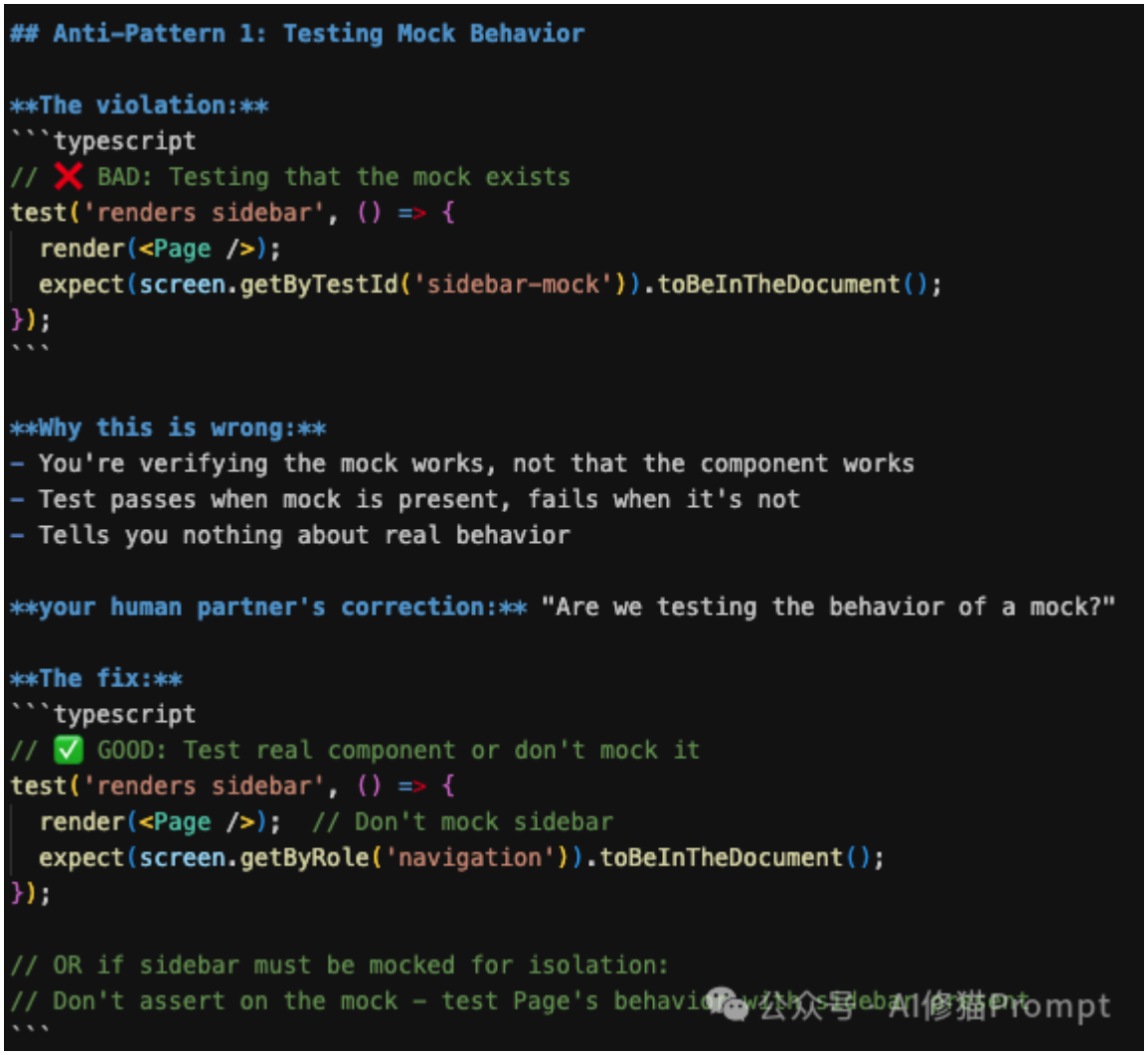
8.test-driven-development（测试驱动开发TDD）

这是Superpowers的**铁律**。技能文档明确指出：“违反规则的字面意思就是违反规则的精神。”



- **红-绿-重构循环**：

    - **红**：必须先写一个失败的测试。
    - **验证红**：必须运行测试并**看着它失败**（Verify Red）。如果测试通过了，说明测试写错了或者功能已存在。
    - **绿**：编写最小量的代码让测试通过。
    - **重构**：在测试通过的前提下清理代码。
- **绝对禁止**：
    - 严禁在没有失败测试的情况下编写生产代码。
    - 如果AI先写了代码再补测试，技能要求它**必须删除代码，重新开始**。
- **测试反模式（Testing Anti-Patterns）**：该技能包含一个附录 `testing-anti-patterns.md`，专门纠正AI常见的错误测试习惯：

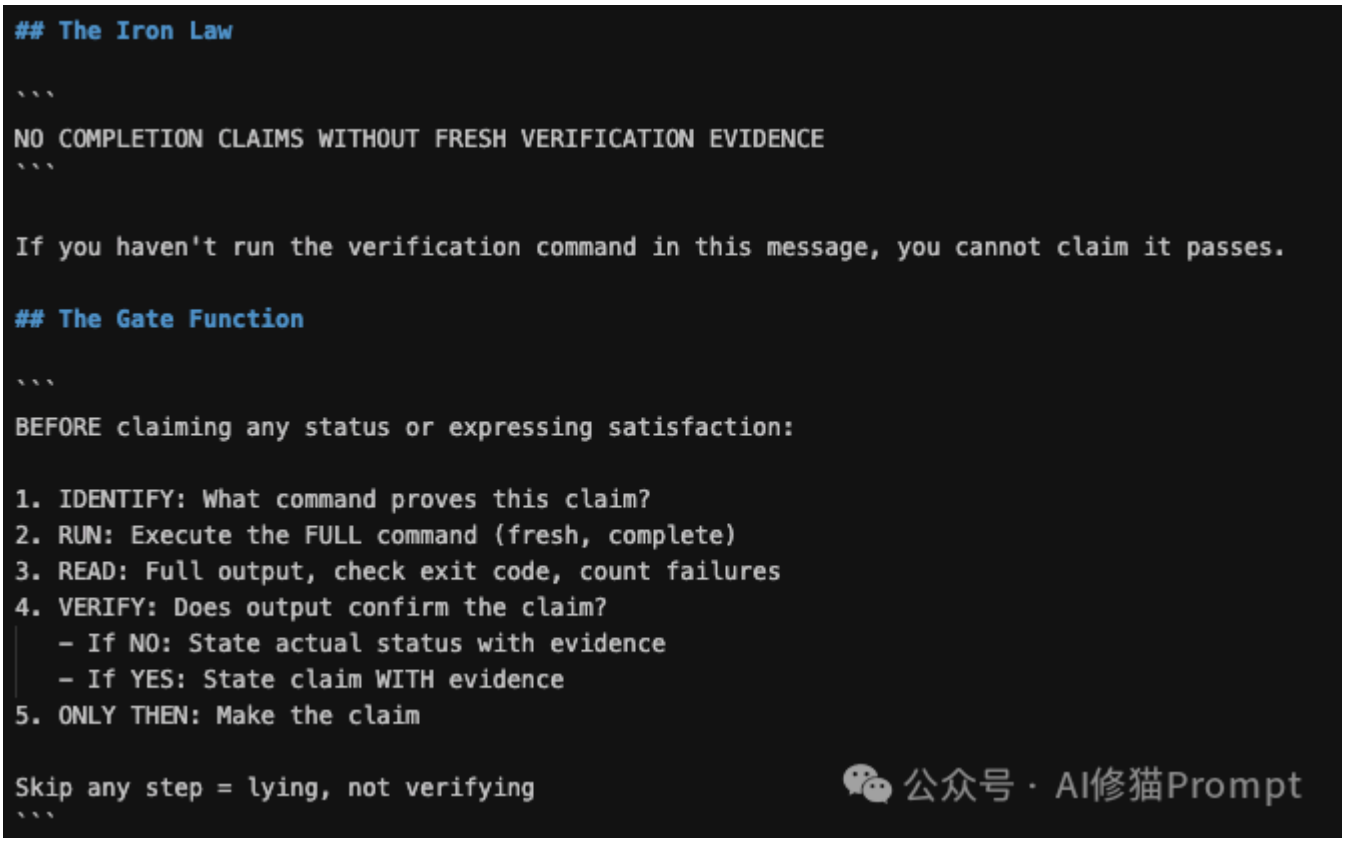


    - **测试Mock而非真实行为**：如果测试只是验证Mock对象被调用了，而没有验证实际业务逻辑，这是无效测试。
    - **在生产代码中添加仅供测试的方法**：严禁为了测试方便而在生产类中添加 `destroy()` 等方法。

- **不理解依赖就Mock**：必须先理解依赖的真实行为（通常通过阅读接口定义），然后再编写Mock。

9.verification-before-completion（完成前验证）

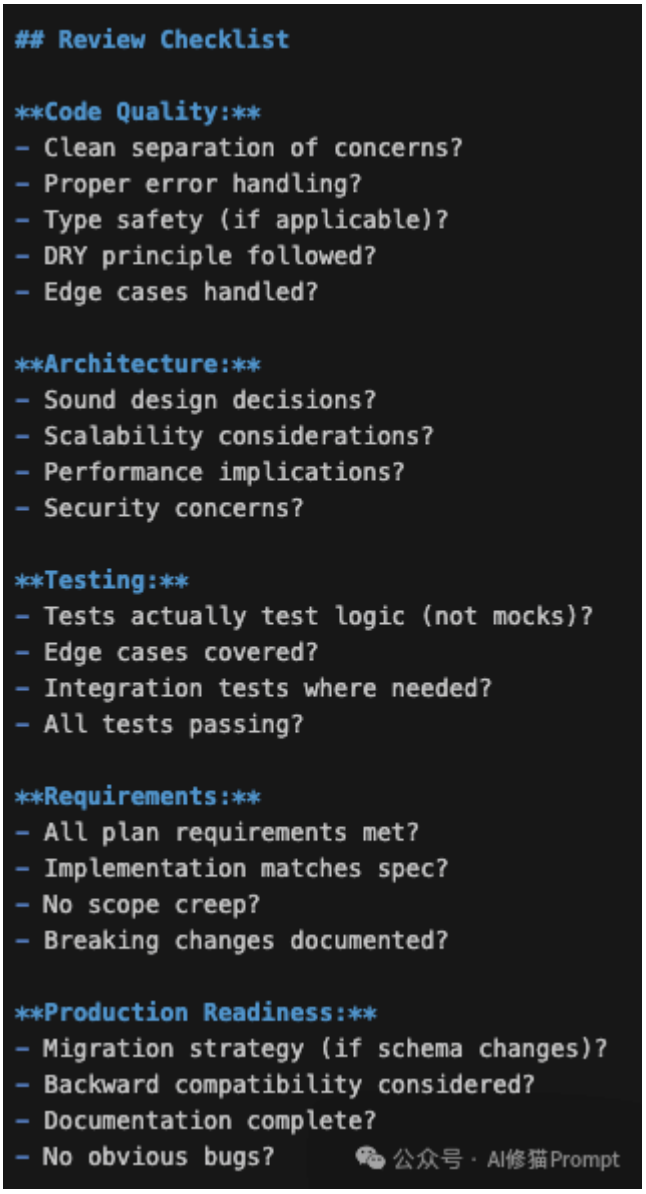
这个技能旨在消除LLM的幻觉和虚假承诺。核心原则是：**证据胜于主张（Evidence over Claims）**。



- **门控机制（Gate Function）**：在Agent声称“任务完成”、“Bug已修复”或“测试通过”之前，它必须：
    1. 确定能证明该主张的命令。
    2. 运行该命令。
    3. 读取完整输出。
    4. **只有在输出证实主张的情况下**，才能向用户报告成功。
- **禁止用语**：严禁使用“应该修复了”、“看起来是对的”、“逻辑上是通的”等模糊语言。没有运行验证命令的结论被视为不诚实。
- **配置变更验证**：特别强调验证配置变更（如切换LLM提供商）时，不能只看“操作成功”，必须验证“输出是否反映了变更”（例如检查响应头中的模型名称）。

10.requesting-code-review（请求代码审查）

- **触发时机**：在任务间隙、功能完成时或合并前。



- **标准化模板**：使用 `code-reviewer.md` 模板，要求审查者对比 `{WHAT_WAS_IMPLEMENTED}` 和 `{PLAN_OR_REQUIREMENTS}`。

- **问题分类**：审查结果必须将问题分为：致命（Critical）、重要（Important）、次要（Minor）。

```
## Output Format

### Strengths
[What's well done? Be specific.]

### Issues

#### Critical (Must Fix)
[Bugs, security issues, data loss risks, broken functionality]

#### Important (Should Fix)
[Architecture problems, missing features, poor error handling, test gaps]

#### Minor (Nice to Have)
[Code style, optimization opportunities, documentation improvements]

**For each issue:**
- File:line reference
- What's wrong
- Why it matters
- How to fix (if not obvious)

### Recommendations
[Improvements for code quality, architecture, or process]
```


- **Git范围**：明确指定 BASE_SHA 和 HEAD_SHA，确保审查范围精确。

11.receiving-code-review（接收代码审查）

这个技能教导AI如何专业地处理反馈，核心原则是：**技术正确性 > 社交礼仪**。

```
## Acknowledging Correct Feedback

When feedback IS correct:
```
✅ "Fixed. [Brief description of what changed]"
✅ "Good catch - [specific issue]. Fixed in [location]."
✅ [Just fix it and show in the code]

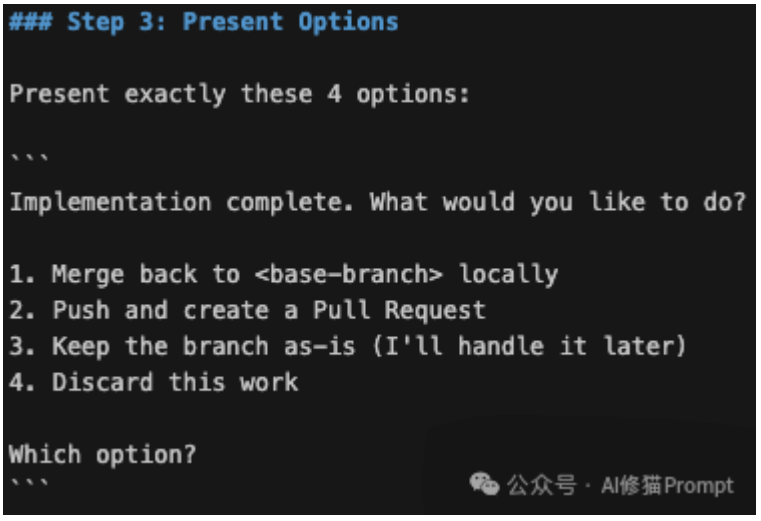
❌ "You're absolutely right!"
❌ "Great point!"
❌ "Thanks for catching that!"
❌ "Thanks for [anything]"
❌ ANY gratitude expression
```

**Why no thanks:** Actions speak. Just fix it. The code itself shows you heard the feedback.

**If you catch yourself about to write "Thanks":** DELETE IT. State the fix instead.
```


- **禁止表演性同意**：严禁LLM说“你说得太对了！”、“极好的建议！”等客套话。

- **先验证后执行**：收到反馈后，必须先在代码库中验证反馈的准确性。
- **YAGNI检查**：如果审查者建议添加“专业”但当前未使用的功能，Agent必须检查代码库，如果确实未被调用，应建议删除（YAGNI）而不是盲目实现。
- **有理有据的反驳**：如果审查意见在技术上不正确或不适合当前架构，Agent必须基于技术理由进行反驳，而不是盲从。

12.finishing-a-development-branch（完成开发分支）

- **最终验证**：在结束开发前，必须再次运行全套测试。
- **标准化选项**：向用户提供四个明确选项：



  1. 本地合并回主分支。
  2. 推送并创建PR。
  3. 保留分支（暂不处理）。
  4. 丢弃工作（需二次确认）。
- **清理**：根据用户的选择，自动清理Git Worktree，保持环境卫生。

第四部分：系统化调试技能

当事情出错时，AI往往倾向于“试错法”（Shotgun Debugging）。Superpowers强制其像福尔摩斯一样工作。

13.systematic-debugging（系统化调试）

- **铁律**：**在找到根本原因之前，禁止尝试修复。**

- **四阶段流程**：

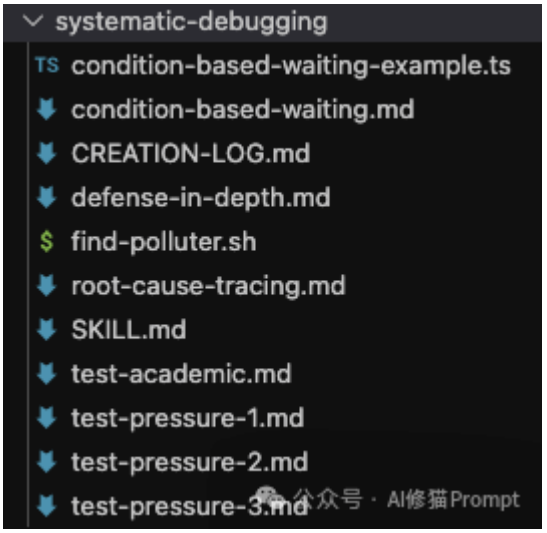
```
## The Four Phases

You MUST complete each phase before proceeding to the next.

### Phase 1: Root Cause Investigation

**BEFORE attempting ANY fix:**

1. **Read Error Messages Carefully**
   - Don't skip past errors or warnings
   - They often contain the exact solution
   - Read stack traces completely
   - Note line numbers, file paths, error codes

2. **Reproduce Consistently**
   - Can you trigger it reliably?
   - What are the exact steps?
   - Does it happen every time?
   - If not reproducible → gather more data, don't guess

3. **Check Recent Changes**
   - What changed that could cause this?
   - Git diff, recent commits
   - New dependencies, config changes
   - Environmental differences

4. **Gather Evidence in Multi-Component Systems**

   **WHEN system has multiple components (CI → build → signing, API → service → database):**

   **BEFORE proposing fixes, add diagnostic instrumentation:**
   ```
   For EACH component boundary:
     - Log what data enters component
     - Log what data exits component
     - Verify environment/config propagation
     - Check state at each layer

   Run once to gather evidence showing WHERE it breaks
   THEN analyze evidence to identify failing component
   THEN investigate that specific component
   ```
```


    1. **根本原因调查**：阅读错误日志，复现问题，检查最近变更。如果是多组件系统，必须添加日志以确定故障发生的边界。
    2. **模式分析**：寻找代码库中类似的、正常工作的代码作为参考。
    3. **假设与测试**：提出单一假设，用最小的改动验证假设。
    4. **实施**：编写一个失败的测试用例（Reproduction），修复它，然后验证。

- **架构质疑**：如果尝试了3次修复都失败了，或者每次修复都引发新问题，Agent必须停止并质疑架构本身，而不是继续打补丁。

该技能还包含几个强大的子工具：

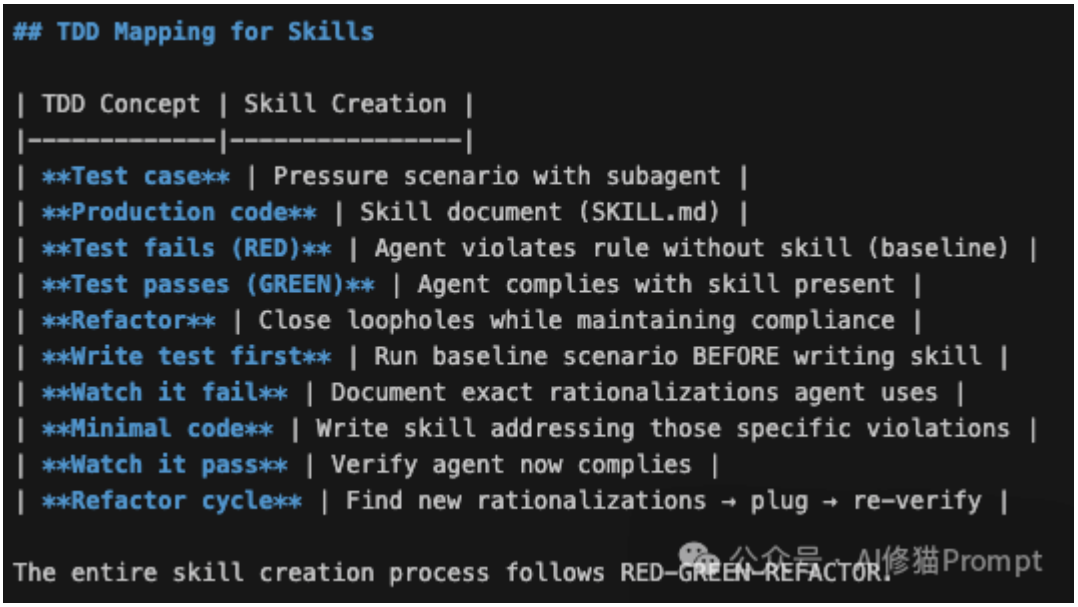


- **`root-cause-tracing`（根因追踪）**：教导AI不要只修复报错的地方（症状），而是要沿着调用栈反向追踪，直到找到错误数据的源头。例如，`git init` 在错误目录执行，不应只在执行处加判断，而应追踪为何传入了错误的路径变量。
- **`defense-in-depth`（纵深防御）**：修复Bug后，要在数据流经的每一层（API入口、业务逻辑、环境卫士、调试日志）都添加校验。目标不仅是修复Bug，而是让该类Bug在结构上不可能再次发生。
- **`condition-based-waiting`（基于条件的等待）**：解决测试不稳定性（Flaky Tests）。严禁使用 `sleep(1000)` 这种随意的延时，必须使用轮询机制等待特定状态或事件（如 `waitForEvent`）。
- **`find-polluter.sh`**：一个二分查找脚本，用于定位是哪个测试用例污染了全局环境（如遗留了文件或进程）。

第五部分：元技能

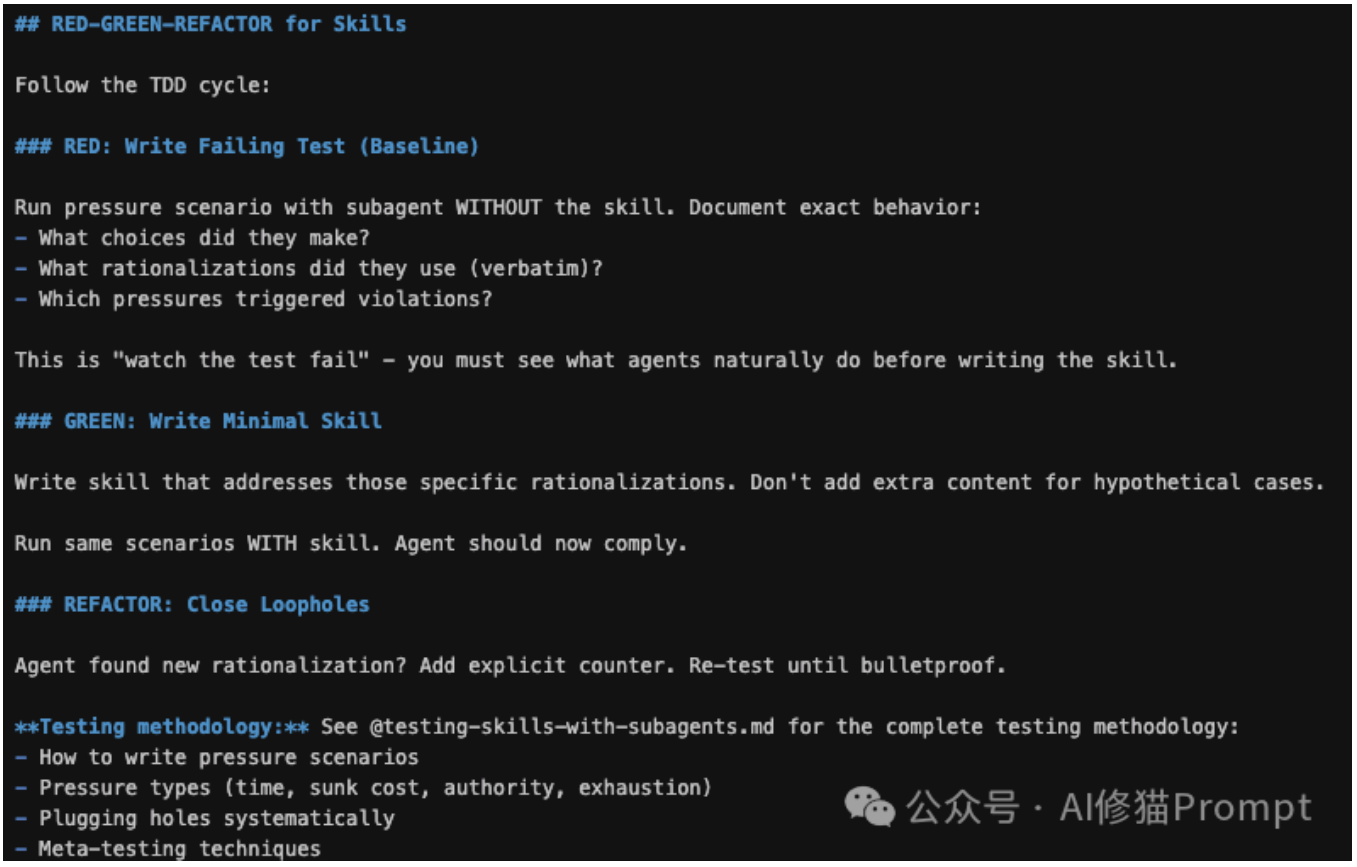
14.writing-skills（编写技能）

既然TDD for Skills是如此核心的哲学，那么如何执行呢？这个Skill就是**执行该哲学的操作手册**。



```
## TDD Mapping for Skills

| TDD Concept | Skill Creation |
|-------------|----------------|
| **Test case** | Pressure scenario with subagent |
| **Production code** | Skill document (SKILL.md) |
| **Test fails (RED)** | Agent violates rule without skill (baseline) |
| **Test passes (GREEN)** | Agent complies with skill present |
| **Refactor** | Close loopholes while maintaining compliance |
| **Write test first** | Run baseline scenario BEFORE writing skill |
| **Watch it fail** | Document exact rationalizations agent uses |
| **Minimal code** | Write skill addressing those specific violations |
| **Watch it pass** | Verify agent now complies |
| **Refactor cycle** | Find new rationalizations → plug → re-verify |

The entire skill creation process follows RED-GREEN-REFACTOR.
```


它是专门为那些**想扩展Superpowers技能库**的开发者准备的。当你告诉LLM“我要写一个新的Skill”时，这个技能会引导您：



```
## RED-GREEN-REFACTOR for Skills

Follow the TDD cycle:

### RED: Write Failing Test (Baseline)

Run pressure scenario with subagent WITHOUT the skill. Document exact behavior:
- What choices did they make?
- What rationalizations did they use (verbatim)?
- Which pressures triggered violations?

This is "watch the test fail" - you must see what agents naturally do before writing the skill.

### GREEN: Write Minimal Skill

Write skill that addresses those specific rationalizations. Don't add extra content for hypothetical cases.

Run same scenarios WITH skill. Agent should now comply.

### REFACTOR: Close Loopholes

Agent found new rationalization? Add explicit counter. Re-test until bulletproof.

**Testing methodology:** See @testing-skills-with-subagents.md for the complete testing methodology:
- How to write pressure scenarios
- Pressure types (time, sunk cost, authority, exhaustion)
- Plugging holes systematically
- Meta-testing techniques
```


- **构建测试场景**：它会强迫您先写出那个“高压剧本”，即模拟LLM最容易偷懒或违规的真实业务场景（例如：紧急修复线上Bug时是否会跳过测试）。
- **捕获合理化借口**：它会帮您记录下LLM在压力下为了绕过规则而编造的具体理由，从而在Skill指令中进行针对性的“补丁”防御。
- **文档结构优化**：它会指导您如何编写 `SKILL.md` 的Frontmatter，特别是提醒您描述字段（Description）应仅包含“触发条件（Use when...）”，严禁包含流程摘要，以防止LLM产生“我已经懂了”的幻觉而跳过阅读正文。

简而言之，这是一个技能工厂，它确保了这套体系具备持续的自我进化能力。

如何部署Superpowers

Superpowers的安装因平台而异。Claude Code拥有内置的插件系统，而Codex和OpenCode则需要通过简单的Agent指令进行引导。

1. Claude Code (通过插件市场)

如果您使用的是Claude Code，请依次执行以下命令：

```
# 注册Superpowers市场
/plugin marketplace add obra/superpowers-marketplace # 安装插件
/plugin install superpowers@superpowers-marketplace
```

安装完成后，输入 `/help`。如果您能看到 `/superpowers:brainstorm`、`/superpowers:write-plan` 和 `/superpowers:execute-plan` 等命令，则说明安装成功。

2. OpenCode

直接向您的OpenCode发送以下指令，它将自动获取并完成剩余的配置工作：

```
Fetch and follow instructions from
https://raw.githubusercontent.com/obra/superpowers/refs/heads/main/.opencode/INSTALL.md
```

3. OpenAI Codex

直接向您的Codex发送以下指令：

```
Fetch and follow instructions from
https://raw.githubusercontent.com/obra/superpowers/refs/heads/main/.codex/INSTALL.md
```

结语

Superpowers项目不仅仅是一组工具脚本，它是一次将人类高级工程智慧“蒸馏”并注入AI的尝试。通过这一系列Agent Skills，项目作者构建了一个**自我纠正的闭环**：

- `brainstorming` 确保方向正确；
- `writing-plans` 确保路径清晰；
- `test-driven-development` 确保代码可用；
- `systematic-debugging` 确保修复有效；
- `subagent-driven-development` 确保过程受控。

对于希望利用AI提升开发效率，同时又不愿牺牲代码质量和工程文化的团队来说，Superpowers提供了一个极具参考价值的范本。它证明了：**限制AI的自由度，反而能最大化它的生产力。**

未来已来，有缘一起同行！